INSTITUTO POLITÉCNICO-IP

DEPARTAMENTO DE ENGENHARIA INFORMÁTICA

LINCENCIATURA EM ENGENHARIA INFORMÁTICA

RELATÓRIO DO PROJECTO

Tema: Sistema de Autenticação com Decoradores e Hash de Senhas

Por:

Guiola André

Filipe Pedro Muanza

Nsimba Alberto Samuel

Paulo Bunga Rodrigues

Curso: Engenharia Informátca
Classe: IIIª Ano
Periódo: Manhã

Orientador

Doc. Moyo Kanivengidio

UÍGE,ABRIL DE 2026

1. Introdução

Este trabalho tem como objectivo o desenvolvimento de um sistema de autenticação básico em Python, simulando os mecanismos de segurança presentes em aplicações reais como sistemas bancários, redes sociais e plataformas corporativas. O sistema foi desenvolvido tendo por base três pilares fundamentais de segurança:

- Protecção de senhas através de hash criptográfico com salt (SHA-256)
- Controlo de acesso a funções sensíveis através de decoradores Python
- Gestão de permissões por nível de utilizador (Role-Based Access Control — RBAC)

O contexto prático escolhido para a simulação foi o de um sistema bancário do Millennium Atlântico, onde diferentes utilizadores possuem diferentes níveis de acesso a informações e operações.

2. OBJECTIVOS

2.1 Objectivo Geral

Implementar um sistema de autenticação funcional que demonstre boas práticas de segurança em aplicações Python.

2.2 Objectivos Específicos

- Implementar hash de senhas com salt aleatório usando o módulo hashlib (SHA-256)
- Criar um decorador @autenticado com suporte a controlo de acesso por roles
- Desenvolver funções protegidas com diferentes níveis de permissão
- Simular um ambiente bancário com dados individuais por utilizador
- Construir uma interface de linha de comando para testar todo o sistema

3. METODOLOGIA E IMPLEMENTAÇÃO

3.1 Hash de Senha com Salt

A protecção das senhas é o primeiro e mais crítico componente do sistema. Em vez de guardar a senha em texto puro, o sistema gera uma "impressão digital" irreversível através do algoritmo SHA-256. Para evitar ataques de rainbow tables, tabelas pré-calculadas de hashes comuns é adicionado um salt único por utilizador antes de aplicar o hash.

O processo de hash segue os seguintes passos:

- Geração de 16 bytes aleatórios com os.urandom(16) para formar o salt
- Concatenação da senha com o salt antes de aplicar o SHA-256
- Armazenamento do hash e do salt no dicionário de utilizadores
- No login, recálculo do hash com o mesmo salt para comparação

3.2 Decorador @autenticado

O decorador é o mecanismo central de controlo de acesso. Funciona como um "segurança na porta": intercepta qualquer chamada a funções protegidas e verifica duas condições antes de permitir a execução.

A estrutura do decorador é organizada em três níveis funcionais:

- autenticado(roles) : recebe a configuração de roles permitidas
- decorador(func) : recebe a função a ser protegida

- wrapper(*args): executa as verificações em tempo real

As verificações realizadas são: (1) se existe um utilizador com sessão activa e (2) se a role do utilizador está na lista de roles autorizadas para aquela função.

3.3 Controlo de Acesso por Roles

O sistema implementa dois níveis de acesso com permissões diferenciadas:

Função	Permissões
ver_dados_sensivel()	user, admin — dados bancários do próprio utilizador
editar_perfil()	user, admin — edição dos dados pessoais
apagar_usuario()	admin apenas — operação destrutiva e irreversível

Esta abordagem aplica o princípio de segurança do Menor Privilégio (Least Privilege): cada utilizador tem acesso apenas ao mínimo necessário para desempenhar a sua função.

3.4 Dados Individuais por Utilizador

Cada utilizador possui dados bancários próprios armazenados no dicionário de utilizadores no momento do cadastro. A função ver_dados_sensivel() recupera dinamicamente os dados do utilizador com sessão activa, garantindo que cada um vê apenas as suas próprias informações.

4. ESTRUTURA DO CÓDIGO

O ficheiro auth_system.py está organizado em seis secções sequenciais:

Secção	Descrição
Secção 1	Armazenamento simulado — dicionários usuarios e current_user
Secção 2	Função hash_senha() com geração de salt e SHA-256
Secção 3	Funções cadastrar(), login() e logout()
Secção 4	Decorador autenticado() com controlo por roles
Secção 5	Funções protegidas com @autenticado

Secção 6	Menu de linha de comando para testes interactivos

5. TESTES E RESULTADOS

O sistema foi testado com os seguintes cenários para validar todos os fluxos:

- Tentativa de acesso sem login : sistema recusa com mensagem clara
- Login com credenciais erradas: sistema recusa sem indicar qual campo está errado
- Login como 'joao' (user) e tentativa de apagar utilizador: acesso negado por role
- Login como 'admin': acesso a todas as funções incluindo apagar_usuario()
- Ver dados sensíveis com utilizadores diferentes : cada um vê os seus próprios dados
- Logout e novo acesso: sessão encerrada correctamente

Todos os cenários produziram os resultados esperados, confirmando o correcto funcionamento do sistema de autenticação e controlo de acesso.

6. PRINCÍPIOS DE SEGURANÇA APLICADOS

- Irreversibilidade do hash : não é possível recuperar a senha original a partir do hash armazenado
- Salt único por utilizador : protege contra ataques de rainbow tables e dicionário
- Mensagem de erro genérica no login: não revela se o erro é no username ou na senha
- Menor Privilégio : cada utilizador acede apenas ao que é necessário para a sua função
- Separação de responsabilidades : cada função tem uma única responsabilidade bem definida

7. CONCLUSÃO

O trabalho permitiu compreender e aplicar na prática os conceitos fundamentais de segurança em aplicações com sistema de login. A implementação do hash com salt demonstrou como proteger credenciais de utilizadores de forma robusta, enquanto o uso de decoradores mostrou uma forma elegante e reutilizável de aplicar controlo de acesso sem repetição de código.

O controlo por roles (RBAC) evidenciou como sistemas reais gerem diferentes níveis de permissão, garantindo que utilizadores comuns não têm acesso a operações administrativas sensíveis. A simulação com dados bancários do Millennium Atlântico tornou o exercício mais próximo de um cenário de aplicação real, facilitando a compreensão dos conceitos abordados.